Extended Thumbnail Control Specification

ADAM ARCHITECTURAL UI BLUEPRINT & FRAMEWORK LAYOUT

1. Executive Summary & Context

This document details the functional, technical, and structural specifications for the **Extended Thumbnail** component (code-named *Adam Thumbnail*). This component is a core user interface primitive designed for high-performance, asset-heavy multimedia layouts built within the **Avalonia UI (11.x+)** ecosystem.

The primary objective of this specification is to separate structural layout mechanics, high-performance interactions, and cross-platform pointer behaviors from individual widget configurations. By establishing a **Zone-Based Template Layering Strategy**, this framework empowers developers to inject arbitrary interactive controls, selection utilities, and custom metadata templates into predefined areas without breaking core container mechanics, virtualization properties, or selection management pipelines.

2. Desktop UI Layout Disambiguation

> **Critical Platform Architectural Note**
>
> Developers coming from legacy Win32, Windows Forms, or VB6 platforms frequently attempt to utilize a `ListView` control to achieve multi-column icon/tile layouts. In modern declarative XAML frameworks like Avalonia UI, a `ListBox` control combined with a custom item layout panel is the correct architectural choice.
>
> Avalonia does not natively feature a `ListView`; instead, its `DataGrid` is strictly designed for structured, spreadsheet-like tabular text columns. To implement a highly custom, free-flowing asset matrix that automatically scales and flows across rows, an items-virtualized `ListBox` container using a horizontal wrapping strategy is mandatory.

3. Architectural Component Mapping

The Extended Thumbnail architecture divides operational responsibilities into two distinct structural elements within the XAML ecosystem:

- **The Master Container (`ListBox` Wrapper):** Manages collection hosting, UI virtualization, high-performance scrolling mechanics, multi-key keyboard navigation rules, and aggregate item selection states.
- **The Item Architecture (`DataTemplate` & `ControlTheme`):** Defines the structural layout, canvas layering bounds, and state-driven animations (such as hover and selection fade-ins) for individual thumbnail items.

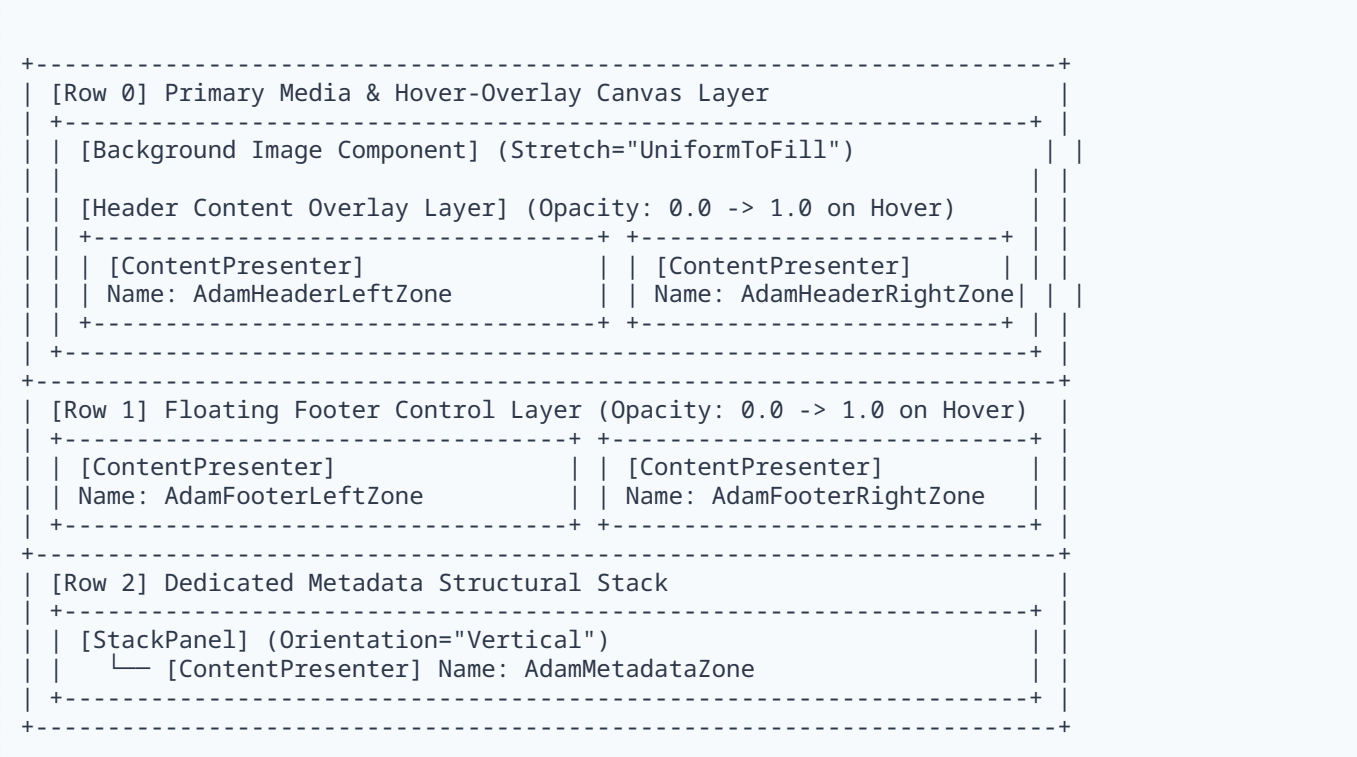
3.1 Grid Flow Configuration

The master container replaces its standard vertical items panel template with a wrap-around layout engine to distribute cards horizontally:

```
<ListBox.ItemsPanel>
    <ItemsPanelTemplate>
        <WrapPanel Orientation="Horizontal"
KeyboardNavigation.TabNavigation="Continue" />
    </ItemsPanelTemplate>
</ListBox.ItemsPanel>
```

4. Structural Layout & Zone Anatomy

Each instance of an Extended Thumbnail component lives inside a parent `ListBoxItem` container. The layout relies on an absolute-depth, layered 3-row grid that decouples media content, active overlay widgets, and bottom descriptive data tags.



4.1 Detailed Layout Zone Specifications

Zone Identifier	Layout Position	Design Intention & Capabilities
AdamHeaderLeftZone	Row 0, Top-Left Align	Designed for view indicators, processing state badges, quick filters, or contextual utility icons.
AdamHeaderRightZone		

Zone Identifier	Layout Position	Design Intention & Capabilities
	Row 0, Top-Right Align	Reserved for overflow menus, system context dropdown triggers (`...`), or deletion commands.
AdamFooterLeftZone	Row 1, Bottom-Left Align	Optimized for persistent toggle triggers, flag symbols, asset bookmarks, or ribbon status controls.
AdamFooterRightZone	Row 1, Bottom-Right Align	Reserved for manual multi-selection flags, checkboxes, or visual item-state indicators.
AdamMetadataZone	Row 2, Full Width Stack	Houses file titles with text-trimming ellipses, byte-size string readouts, pixel dimensions, and category descriptors.

5. Interactive States & Visual State Manager (VSM)

The framework orchestrates mouse and touch responses globally via native XAML pseudo-classes. This isolates your custom controls from tracking hover flags manually.

5.1 Hover Mechanics (`:pointerover`)

By default, to maximize user focus on raw media content, all elements bound within the Header and Footer overlay layers are set to `Opacity="0.0"`. When the pointer crosses into the bounding box of the `ListBoxItem`:

- The item shifts to the `:pointerover` pseudo-class.
- An internal animation pipeline changes the target overlay layers to `Opacity="1.0"` smoothly over 0.2 seconds using an `EaseInOut` curve.
- The outer decorative border shifts to a subtle highlight tone.

5.2 Selection Mechanics (`:selected`)

When an item is committed to the selection array (via tap, click, or marquee bounding-box overlap):

- The control enters the `:selected` state, locking the outer bounding frame to the system's primary accent highlight color.
- Any custom validation checkboxes mapped into `AdamFooterRightZone` receive structural notifications to match their selection state visual trees.

6. Implementation Blueprint: Ready-to-Paste Production Code

The following layout model provides the precise XAML blueprint necessary to implement the Extended Thumbnail framework within an Avalonia application.

6.1 Core Layout XAML Template

```
<UserControl xmlns="https://github.com/avaloniaui"
             xmlns:x="http://schemas.microsoft.com/winfx/2000/xaml"
             x:Class="Adam.UI.Controls.ExtendedThumbnail">

    <UserControl.Styles>
        <!-- Framework Opacity Animation States -->
        <Style Selector="ListBoxItem Grid#OverlayContainer">
            <Setter Property="Opacity" Value="0.0"/>
            <Setter Property="Transitions">
                <Transitions>
                    <DoubleTransition Property="Opacity" Duration="0:0:0.2"
Easing="CurveEaseInOut"/>
                </Transitions>
            </Setter>
        </Style>

        <!-- Hover Transition Mapping -->
        <Style Selector="ListBoxItem:pointerover Grid#OverlayContainer">
            <Setter Property="Opacity" Value="1.0"/>
        </Style>

        <!-- Decorative Outer Border States -->
        <Style Selector="ListBoxItem">
            <Setter Property="BorderBrush" Value="Transparent"/>
            <Setter Property="BorderThickness" Value="1"/>
            <Setter Property="Padding" Value="6"/>
        </Style>
        <Style Selector="ListBoxItem:pointerover">
            <Setter Property="BorderBrush" Value="#3182CE"/>
        </Style>
    </UserControl.Styles>

    <ListBox Name="AdamThumbnailGrid">
        <ListBox.ItemsPanel>
            <ItemsPanelTemplate>
                <WrapPanel Orientation="Horizontal" />
            </ItemsPanelTemplate>
        </ListBox.ItemsPanel>

        <ListBox.ItemTemplate>
            <DataTemplate>
                <Border Background="#FFFFFF" CornerRadius="4" BoxShadow="0 1 3 0
#20000000" Margin="6" Width="220">
                    <Grid RowDefinitions="Auto, Auto, Auto">
```

```
                        <!-- Row 0: Base Media & Floating Header Zone -->
                        <Panel Grid.Row="0" Height="130" ClipToBounds="True">
                            <!-- Background Media Layer -->
                            <Image Source="{Binding ImageSource}"
Stretch="UniformToFill"/>

                            <!-- Contextual Overlay Grid Canvas -->
                            <Grid x:Name="OverlayContainer" ColumnDefinitions="*, *"
VerticalAlignment="Top" Padding="6">
                                <ContentPresenter Grid.Column="0" Content="{Binding
HeaderLeftContent}" HorizontalAlignment="Left"/>
                                <ContentPresenter Grid.Column="1" Content="{Binding
HeaderRightContent}" HorizontalAlignment="Right"/>
                            </Grid>
                        </Panel>

                        <!-- Row 1: Floating Footer Control Layer -->
                        <Grid Grid.Row="1" ColumnDefinitions="*, *" Padding="6, 4">
                            <ContentPresenter Grid.Column="0" Content="{Binding
FooterLeftContent}" HorizontalAlignment="Left"/>
                            <ContentPresenter Grid.Column="1" Content="{Binding
FooterRightContent}" HorizontalAlignment="Right"/>
                        </Grid>

                        <!-- Row 2: Metadata Core Stack -->
                        <StackPanel Grid.Row="2" Orientation="Vertical" Padding="8, 2,
8, 8">
                            <ContentPresenter Content="{Binding MetadataContent}"/>
                        </StackPanel>

                    </Grid>
                </Border>
            </DataTemplate>
        </ListBox.ItemTemplate>
    </ListBox>
</UserControl>
```

7. Data Binding Contracts & ViewModels

To preserve decoupling, backend architectural implementations can bind specific visual elements directly to individual asset views. The underlying container model requires tracking the following base contractual variables:

- `ImageSource` (IBitmap): The target high-resolution background asset representation.
- `HeaderLeftContent` (object): Content element containing custom tools or status indicators.
- `HeaderRightContent` (object): Content element hosting the primary context action commands.

- `FooterLeftContent` (object): Content target for state management controls (e.g., bookmark switches).
- `FooterRightContent` (object): Container target managing selection checkboxes or toggle tags.
- `MetadataContent` (object): Base target binding custom multiline labels, file descriptions, size parameters, or text stacks.